



SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

T.C.SAKARYA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

Bilgisayar Mühendisliği Bölümü

BİLGİSAYAR MİMARİSİ VE ORGANİZASYONU

MİKROPROGRAMLANMIŞ KONTROL DEVRESİ TASARIMI

Grup No	Grup 6
Üye 1	Muhammed Eren KENDİR – 22010903097
Üye 2	Ayhan SOYDANER – 25010903031
Üye 3	Arda MUTLUŞAHİNOĞLU – 25010903021
Üye 4	Göktuğ ALAN – 25010903150

İÇİNDEKİLER

1. ÖZET

2. GİRİŞ

3. TEORİK ARKA PLAN

- o 3.1. Mano Temel Bilgisayarı
- o 3.2. Hardwired Kontrol Devresi
- o 3.3. Mikroprogramlanmış Kontrol Mimarisi

4. TASARIM

- o 4.1. Veri Yolu Tasarımı
 - 4.1.1. Yazmaç Modülleri
 - 4.1.2. Aritmetik ve Mantıksal Birim (ALU)
 - 4.1.3. Ana Bellek (RAM)
 - 4.1.4. Ortak Veri Yolu (Common Bus)
 - 4.1.5. Tasarım Kararları
- o 4.2. Mikrokomut Formatı
- o 4.3. Kontrol Belleği ve Mikroprogram
- o 4.4. Performans Kriterleri ve Sınırlamalar

5. UYGULAMA VE SİMÜLASYON

- o 5.1. Verilog Implementasyonu
 - 5.1.1. Modül Hiyerarşisi
 - 5.1.2. Veri Yolu Katmanı Detayları
 - 5.1.3. Kontrol Birimi Katmanı Detayları
 - 5.1.4. Kontrol Birimi Dallanma Mantığı
 - 5.1.5. Üst Modül Entegrasyonu
 - 5.1.6. Tasarım Kararları ve Gerekçeleri
- o 5.2. Test Senaryoları ve Sonuçlar
 - 5.2.1. Birim Test: Kontrol Birimi (control_unit_tb.v)
 - 5.2.2. Entegrasyon Testi: Mano Bilgisayarı

6. SONUÇ

7. KAYNAKLAR

EK A – GÖREV DAĞILIMI

1. ÖZET

Bu çalışmada Morris Mano tarafından tanımlanmış temel bilgisayar mimarisinin donanımsal (hardwired) kontrol devresi, mikroprogramlanmış kontrol mimarisi ile yeniden tasarlanmıştır. Tasarım, Verilog donanım tanımlama dili kullanılarak gerçekleştirilmiştir; Xilinx Vivado ortamında simüle edilmiştir. Geliştirilen sistemde kontrol birimi, 128 × 20 bitlik bir kontrol belleğinden okunan mikrokomutlar aracılığıyla çalışmakta; bu sayede komut seti değişikliklerine esnek biçimde olanak tanımaktadır. Test sonuçları, tasarlanan mikroprogramlanmış kontrol biriminin Mano bilgisayarının temel bellek referanslı komutları doğru biçimde yürüttüğünü doğrulamaktadır. [1] (PÇ1, PÇ5)

2. GİRİŞ

Bilgisayar mimarisi ve organizasyonu alanında kontrol birimi, bir işlemcinin her saat darbesinde hangi işlemlerin gerçekleştirileceğini belirleyen merkezi birimdir. Kontrol biriminin doğru ve verimli tasarımı, sistemin bütünsel performansını doğrudan etkilemektedir. Tarihsel süreç incelendiğinde kontrol birimlerinin iki temel yaklaşımla gerçekleştirildiği görülebilir: donanımsal (hardwired) kontrol ve mikroprogramlanmış kontrol. Donanımsal kontrolde her kontrol sinyali, sisteme özgü kombinasyonel lojik devrelerle doğrudan üretilir; bu yöntem yüksek hız sağlamakla birlikte komut seti büyüdükçe tasarım karmaşıklığını hızla artırmaktadır. [1][2] (PÇ1)

Mikroprogramlanmış kontrol, Maurice Wilkes tarafından 1951 yılında önerilmiş olup kontrol sinyallerini bir ROM yapısı üzerine kaydedilmiş mikrokomutlar aracılığıyla üretmeyi esas almaktadır. [1][3]

Bu yaklaşım, kontrol mantığını donanım katmanından soyutlayarak komut setinin güncellenmesini yalnızca bellek içeriği değiştirilerek uygulanabilir kılmaktadır. Mano temel bilgisayarı, doğası gereği sade ve öğretici bir mimari olup hem hardwired hem de mikroprogramlanmış kontrol yaklaşımlarını karşılaştırmalı olarak incelemek için ideal bir referans platformu sunmaktadır. [1] (PÇ5)

Bu proje kapsamında Mano bilgisayarının hardwired kontrol devresi, mikroprogramlanmış kontrol mimarisine dönüştürülmüştür. Projenin hedefleri dört ana başlık altında toplanmaktadır: (i) Mano bilgisayarının tüm veri yolu bileşenlerini Verilog ile modellemek, (ii) 20-bitlik mikrokomut formatı ve 128 konumlu kontrol belleğine dayalı bir mikroprogramlanmış kontrol birimi tasarlamak ve gerçeklemek, (iii) simülasyon tabanlı testlerle tasarımın doğruluğunu kapsamlı biçimde kanıtlamaktır [1] (PÇ5)

Proje dört kişilik bir grup tarafından yürütülmüş olup görev dağılımı Ek A'da ayrıntılı şekilde sunulmaktadır. Çalışma sürecinde grup üyeleri düzenli

iletiřim kurarak tasarımı kararlarını ortak tartıřmalar sonucunda řekillendirmiř; veri yolu ile kontrol birimi arayüz tanımları öncelikli olarak belirlenerek paralel geliřtirme süreci mümkün kılınmıřtır. (PÇ13)

3. TEORİK ARKA PLAN

3.1. Mano Temel Bilgisayarı

Mano temel bilgisayarını (Mano Basic Computer), Morris Mano'nun 'Computer System Architecture' adlı eserinde tanımlanan ve bilgisayar mimarisi eęitiminde referans alınan, sade yapısıyla tüm temel mimariyel kavramları barındıran bir öğretili iřlemci modelidir. Mimari 16 bitlik sözcük uzunluęuna sahip olup 4096 adreslenebilir bellek konumundan ($4096 \times 16 \text{ bit} = 8 \text{ KB}$) oluşur. Bellek adresleri 12 bit genişliğinde olmakla birlikte her bellek sözcüęü 16 bit bilgi taşımaktadır. [1] (PÇ1)

Mano bilgisayarında kullanılan yazmaçların tamamını Tablo 3.1'de özetlenmiřtir. Sistemin veri yolu, sekiz farklı kaynaęı 3-bitlik seçim sinyali (S2, S1, S0) ile yöneten 16-bitlik ortak bir veri yolu (common bus) üzerine inşa edilmiřtir. Aritmetik ve mantıksal iřlemlerin tamamını 16-bitlik AC (Accumulator) yazmacı üzerinde gerçekleştirilirken bellek okuma/yazma iřlemleri 12-bitlik AR (Address Register) aracılıęıyla yürütölmektedir. [1] (PÇ1)

Yazmaç	Geniřlik (bit)	İřlev
AC (Accumulator)	16	Aritmetik/mantıksal iřlem sonuçları
DR (Data Register)	16	Bellekten okunan ya da yazılacak veri
AR (Address Register)	12	Bellek eriřim adresi
PC (Program Counter)	12	Sonraki komutun bellekteki adresi
IR (Instruction Reg.)	16	Çekilen komut kodu
TR (Temp. Register)	16	Geçici veri depolama (BSA, ISZ)
INPR (Input Register)	8	Giriř aygıtından okunan veri
OUTR (Output Register)	8	Çıkıř aygıtına gönderilen veri
E (Extended Bit)	1	Tařıma/tařma biti (carry/overflow)

Tablo 3.1. Mano Temel Bilgisayarı Yazmaçları

Komut seti ü. kategoriden oluşur. Bellek referanslı komutlar (memory-reference), 16-bitlik sözcüęün ilk üç biti (IR[14:12]) komut kodunu, 15. biti (IR[15]) dolaylı/doęrudan adresleme bitini ve son 12 biti (IR[11:0]) etkin adresi barındıran formata sahiptir; AND, ADD, LDA, STA, BUN, BSA ve ISZ bu kategoride yer alır. Register referanslı komutlar (register-reference), IR[15:12] = '1111 0' şeklinde kodlanır ve CLA, CLE, CMA, CME, CIR, CIL, INC, SPA, SNA, SZA, SZE, HLT komutlarını kapsar. G/C komutları (I/O) ise IR[15:12] = '1111 1' ile tanımlanan INP, OUT, SKI, SKO, ION, IOF komutlarından oluşur. [1] (PÇ1)

Komut çalışma döngüsü T0-T7 zaman adımlarından oluşan bir dizi sayıcı (sequence counter - SC) tarafından yönetilir. Her saat darbesi SC'yi bir artırır; SC, kontrol sinyali SC = 0 üretilerek sıfırlanır ve döngü yeniden T0 adımına döner. Bir komut tam olarak kaç T adımıyla tamamlandığına bağlı olarak SC farklı noktalarda temizlenmektedir; bu durum donanımsal kontrolün temel mekanizmasını oluşturmaktadır. [1] (PÇ1)

3.2. Hardwired Kontrol Devresi

Hardwired kontrol devresi, kontrol sinyallerini donanımsal düzeyde AND-OR kapı ağları ve flip-floplar aracılığıyla doğrudan üreten bir yaklaşımdır. Mano bilgisayarının orijinal hardwired tasarımında kontrol birimi; 4-bitlik dizi sayıcı (SC), 3-to-8 komut çözücüsü (D0-D7), ve çok sayıda kombinasyonel lojik ifadeden oluşur. Her kontrol sinyali, SC'nin ürettiği T zaman damgası, IR içeriği ve çeşitli durum bayraklarıyla (E, FGI, FGO, IEN, R) tanımlanan boolean ifadesiyle belirlenir. [1] (PÇ1)

Getirme (fetch) döngüsü üç T adımıyla tamamlanır ve tüm komutlar için ortaktır. Register Transfer Language (RTL) gösterimi aşağıda verilmiştir:

```
T0 : AR <- PC
T1 : IR <- M[AR], PC <- PC + 1
T2 : D0,...,D7 <- Decode[IR(14:12)], AR <- IR(11:0), I <- IR(15)
```

Şekil 3.2. Getirme Döngüsü RTL Gösterimi

T2 adımının ardından komuta ve adresleme moduna göre farklı yollar izlenir. Doğrudan adresleme modunda (I = 0) AND komutu örneği için yürütme döngüsü şunları kapsar:

```
D0 . I' . T3 : DR <- M[AR]
D0 . I' . T4 : AC <- AC ^ DR, SC <- 0
```

Şekil 3.3. AND Komutu (Doğrudan Adresleme) RTL Gösterimi

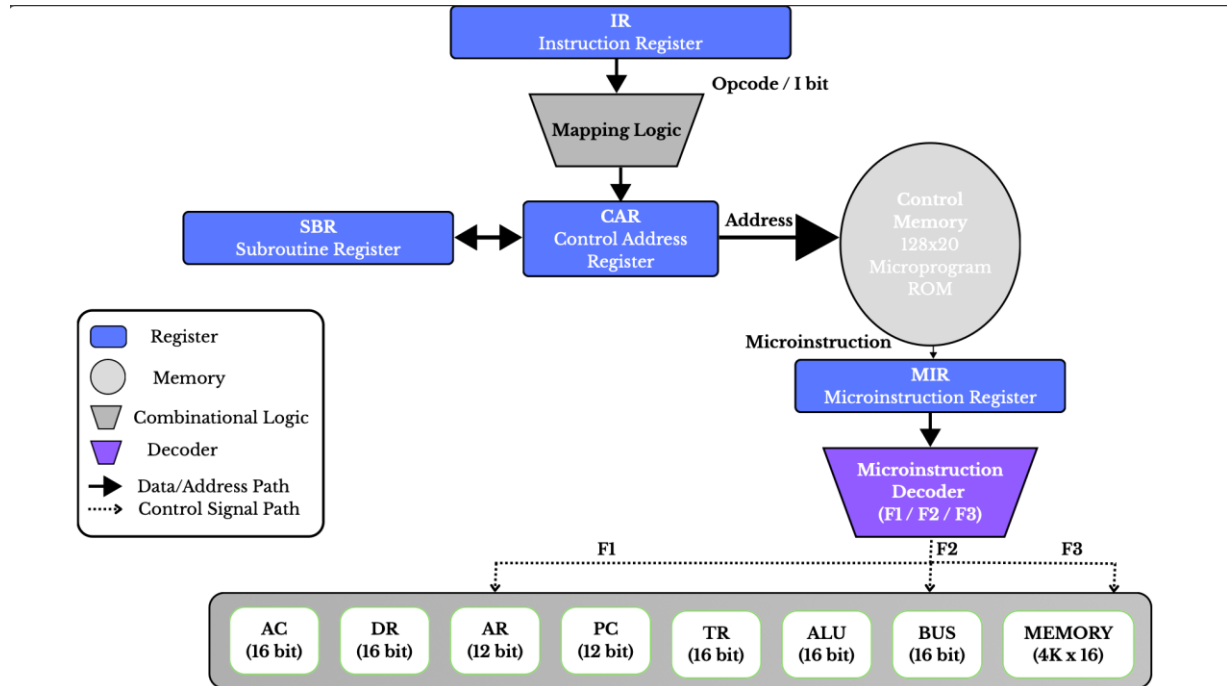
Dolaylı adresleme modunda (I = 1) ise T3'te AR <- M[AR] işlemiyle gerçek etkin adres çözümlenerek T4 ve T5 adımlarında yürütme tamamlanır; bu durum hardwired kontrolde ek T adımlarının kombinasyonel lojikle yönetilmesini zorunlu kılar. Her yeni komut eklendikçe bu boolean denklem ağları büyümekte ve tasarım, düzenlenmesi ve test edilmesi gittikçe zorlaşan monolitik bir yapıya dönüşmektedir. [1] (PÇ5)

Hardwired kontrolün temel avantajı, kontrol sinyallerinin doğrudan donanımla üretilmesi nedeniyle komut başına minimum saat darbesi gerektirmesi ve yüksek çalışma frekansı sağlamasıdır. Bununla birlikte en önemli dezavantajları şunlardır: komut seti büyüdükçe lojik kapı sayısı hızla artmakta ve yeniden tasarım maliyeti yükselmektedir; herhangi bir komutun eklenmesi ya da değiştirilmesi donanım düzeyinde kapsamlı değişiklik gerektirmektedir. Bu kısıt, mikroprogramlanmış kontrolün geliştirilmesine yol açan temel mühendislik problemi olarak değerlendirilmektedir. [1][2] (PÇ5)

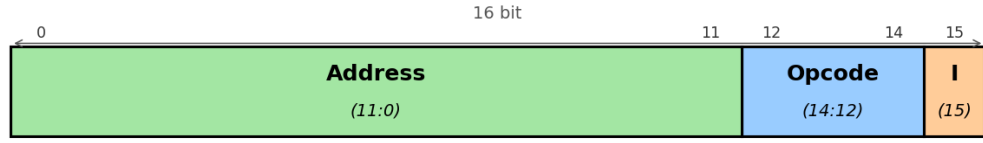
3.3. Mikroprogramlanmış Kontrol Mimarisi

Mikroprogramlanmış kontrol kavramı, Maurice Wilkes tarafından 1951 yılında Manchester Üniversitesi konferansında 'The best way to design an automatic calculating machine' başlığıyla yayımlanan çalışmayla ortaya konulmuştur. Temel fikir şunları kapsamaktadır: her makine komutu, bir kontrol belleğinde (control memory) saklanan bir dizi mikrokomutla (microinstruction) temsil edilir; kontrol birimi her saat darbesinde bu bellekten bir mikrokomut okuyarak ilgili kontrol sinyallerini üretir. Mikrokomutlar bir ROM yapısında depolandığı için komut seti değişikliği donanım değiştirme gerektirmeden yalnızca ROM içeriği güncellenerek uygulanabilir. [3] (PÇ1)

Mikroprogramlanmış kontrol biriminin temel bileşenleri Şekil 3.1'de gösterilmiştir. Kontrol Adres Yazmacı (CAR - Control Address Register), okunacak bir sonraki mikrokomutun adresini tutan 7-bitlik bir yazmacıdır; 7-bitlik CAR ile 128 farklı mikrokomut konumu adreslenebilmektedir. Alt Program Yazmacı (SBR - Subroutine Register), alt program çağrısı (CALL) anında bir sonraki adresin saklandığı 7-bitlik bir yazmacıdır ve dolaylı adresleme subroutine'inden dönüşte kullanılır. Mikrokomut Yazmacı (Microinstruction Register), kontrol belleğinden okunan 20-bitlik mikrokomutun geçici olarak tutulduğu yazmaçdır. [1][3] (PÇ1)

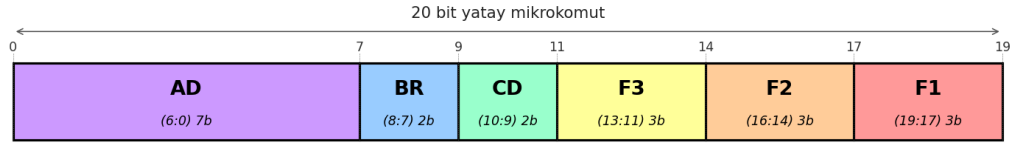


Şekil 3.4. Mikroprogramlanmış Kontrol Birimi Blok Diyagramı



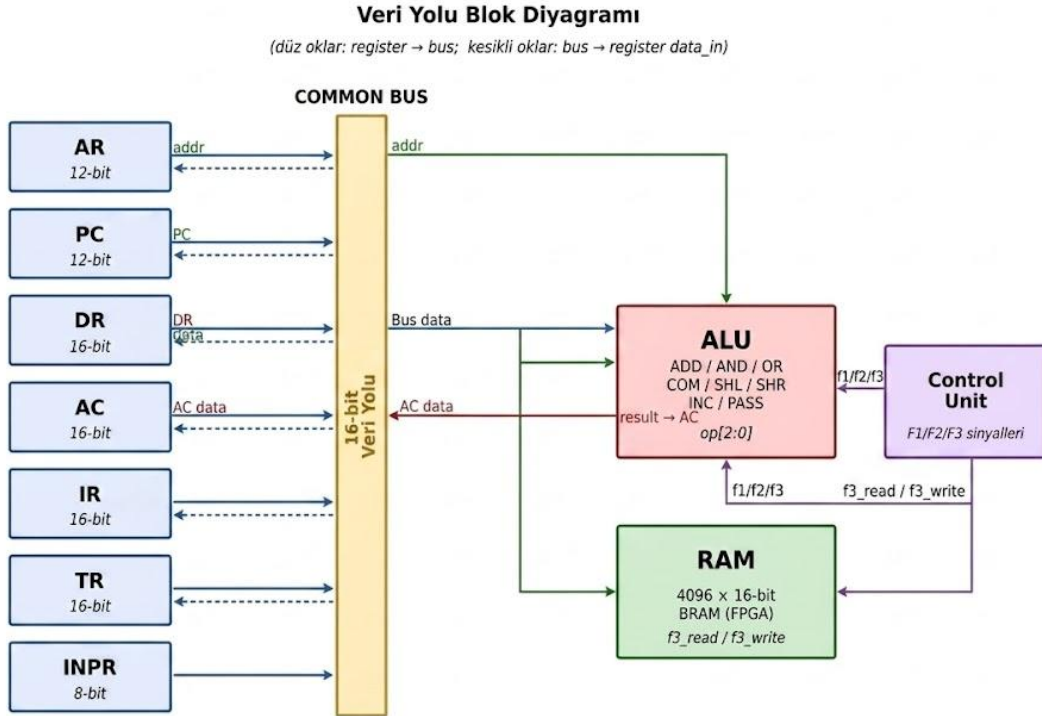
I = Dolaylı adresleme biti | Opcode = İşlem kodu (3 bit, 8 komut) | Address = 12-bit bellek adresi

Şekil 3.5. Komut Yazmacı (IR) Bit Formatı



F1/F2/F3: mikroişlem alanları | CD: koşul (U/I/S/Z) | BR: dallanma (JMP/CALL/RET/MAP) | AD: hedef adres

Şekil 3.6. 20-bit Yatay Mikrokomut Bit Formatı



Şekil 3.7. Veri Yolu (Datapath) Blok Diyagramı

Bu tasarımda benimsenen mikrokomut formatı 20 bit genişliğinde olup altı alana bölünmüştür. F1 (bit 19-17, 3 bit) alanı ALU ve AC üzerindeki mikro-islemleri (ADD, CLRAC, INCAC, DRTAC, ANDAC, COMAC, CLRE), F2 (bit 16-14, 3 bit) alanı ikincil ALU işlemleri, kaydırma ve yazmac transferlerini (SUB, OR, SHL, SHR, INCPC, ARTPC, COME), F3 (bit 13-11, 3 bit) alanı ise bellek erişimi ve yazmac transferlerini (READ, WRITE, PCTAR, IRTAR, ACTDR, INCDR, DRTIR) kodlar. CD (bit 10-9, 2 bit) alanı dallanma koşulunu (00: koşulsuz, 01: IR[15] dolaylı adresleme biti, 10: AC[15] işaret biti, 11: AC = 0 sıfır bayrağı), BR (bit 8-7, 2 bit) alanı dallanma tipini (00: JMP, 01: CALL, 10: RET, 11: MAP) ve AD (bit 6-0, 7 bit) alanı hedef adresi belirtir. Toplam 20 bit genişliğindeki bu format, bir mikrokomutta F1, F2 ve F3 alanlarından birden fazlasının eş zamanlı aktif olabilmesine olanak tanıyan yatay mikroprogramlama (horizontal microprogramming) anlayışına uygundur. [1] (PÇ1)

Alan	Bit Konumu	Genişlik	İslev
F1	19-17	3 bit	ALU ve AC mikro-islemleri (ADD, CLRAC, INCAC, DRTAC, ANDAC, COMAC, CLRE)
F2	16-14	3 bit	İkincil ALU işlemleri ve transferler (SUB, OR, SHL, SHR, INCPC, ARTPC, COME)
F3	13-11	3 bit	Bellek erişimi ve yazmac transferleri (READ, WRITE, PCTAR, IRTAR, ACTDR, INCDR, DRTIR)
CD	10-9	2 bit	Dallanma koşulu (koşulsuz / IR[15] / AC[15] / AC=0)
BR	8-7	2 bit	Dallanma tipi (JMP / CALL / RET / MAP)
AD	6-0	7 bit	Hedef adres (128 konum için)

Tablo 3.8. 20-Bitlik Mikrokomut Formatı

Komut yürütme döngüsü, mikroprogramlanmış yaklaşımda da üç aşama olarak tanımlansa da her aşama artık donanımsal lojik yerine kontrol belleğindeki mikrokomutlar tarafından yürütülmektedir. Getirme aşaması (fetch) 0x40 adresinden başlayan dört mikrokomut adımını kapsar ve tüm makine komutları için ortaktır:

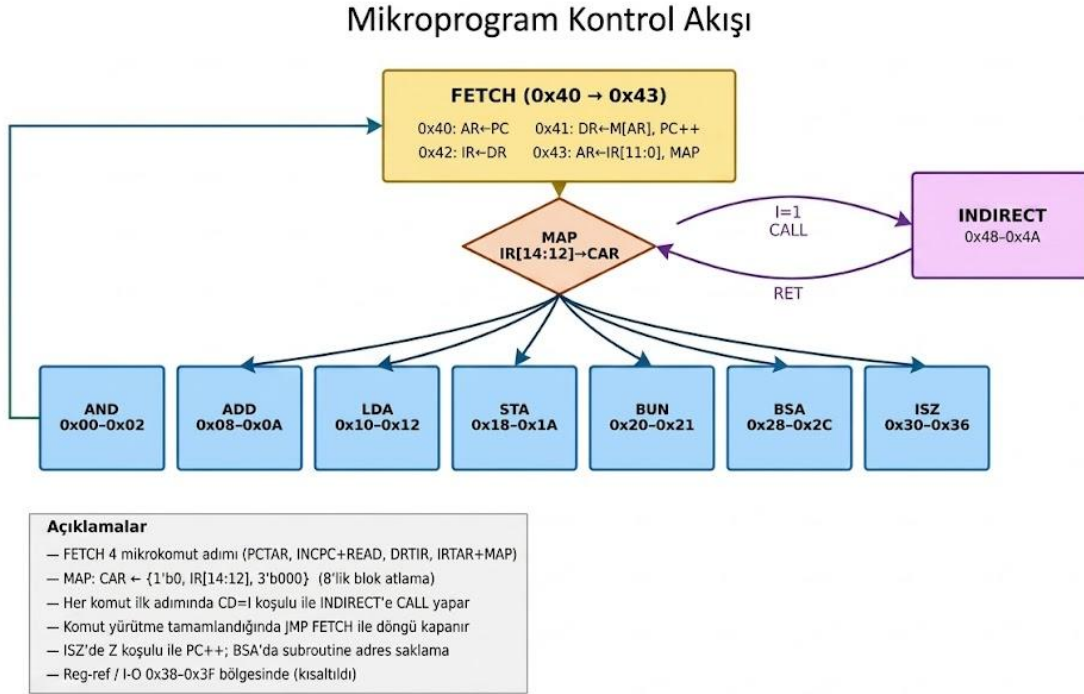
```

0x40:  AR <- PC                (F3=PCTAR, BR=JMP, AD=0x41)
0x41:  DR <- M[AR], PC <- PC+1 (F2=INCPC, F3=READ, BR=JMP, AD=0x42)
0x42:  IR <- DR                (F3=DRTIR, BR=JMP, AD=0x43)

```


0x43: AR <- IR[11:0], MAP (F3=IRTAR, BR=MAP)
MAP: CAR <- {1'b0, IR[14:12], 3'b000}

Şekil 3.9. Fetch Döngüsü Mikroprogram Adımları



Şekil 3.10. Mikroprogram Kontrol Akış Diyagramı

MAP işlemi, IR'nin 14. ile 12. bitleri arasındaki 3-bitlik komut kodunu CAR'a yükleyerek (CAR <- '0' & IR[14:12] & '000') her komutun kontrol belleğindeki kendi mikroprogram bloğuna otomatik olarak yönlendirilmesini sağlar. Bu mekanizma sayesinde tasarımcı, her komut için 8 mikrokomut konumuna kadar kullanabileceği bir yürütme bloğu elde etmektedir. [1] (PÇ5)

Dolaylı adresleme subroutine'i ise 0x48 adresinde konumlandırılmış olup IR[15] = 1 olduğunda CALL komutuyla çağrılır, SBR'a bir sonraki adres kaydedilir ve M[AR]'dan gerçek efektif adres çözümlendikten sonra RET komutuyla araya geri donülür. Bu subroutine mekanizması, tüm bellek referanslı komutların aynı dolaylı adresleme kodunu paylaşmasına olanak tanımakta ve mikroprogram belleğinin verimli kullanılmasını desteklemektedir. [1] (PÇ1, PÇ5)

Yatay mikroprogramlama (horizontal microprogramming) anlayışıyla tasarlanan bu formatta, bir mikrokomut içinde F1, F2 ve F3 alanları eş zamanlı aktif olabilmekte; örneğin aynı anda hem bellek okuma (F3) hem de PC artırma (F2) işlemleri birlikte kodlanabilmektedir. Bu yaklaşım, komut başına gerekli mikrokomut sayısını azaltarak sistemin komut işlem hızını (throughput)

artırmaktadır. Diğer yaklaşım olan dikey mikroprogramlamada (vertical microprogramming) ise her mikrokod tek bir mikro-işlemi kodlar ve daha dar bit genişliği kullanılır; ancak bu durumda daha fazla mikrokod adımı gerekmekte ve kontrol belleğinin derinliği artmaktadır. Bu tasarımda yatay yaklaşımın benimsenmesinin temel gerekçesi, Mano kod setinin görece sade yapısı nedeniyle genişlik artışının kabul edilebilir düzeyde kalmasıdır. [1][2] (PÇ1, PÇ5)

Ozellik	Hardwired	Mikroprogramlanmış
Kontrol sinyali üretimi	Kombinasyonel lojik kapılar	Kontrol belleğindeki mikrokodlar
Hız	Yüksek (min. saat darbesi)	Orta (bellek gecikme süresi var)
Tasarım karmaşıklığı	Buyuk kod setinde çok yüksek	Görece düşük, modüler
Esneklik	Düşük (donanım değişimi gerekir)	Yüksek (ROM güncelleme yeterli)
Kod seti güncelleme	Donanım yeniden tasarımı	Bellek içeriği güncelleme
Tipik kullanım alanı	RISC, yüksek performanslı	CISC, esnek mimari, eğitim

Tablo 3.11. Hardwired ve Mikroprogramlanmış Kontrol Karşılaştırması

4. TASARIM

4.1. Veri Yolu Tasarımı

Veri yolu tasarımı, Bolum 3.1'de tanımlanan tüm yazmaçları, aritmetik ve mantıksal birim (ALU), 4096 x 16-bitlik senkron RAM belleğini ve ortak veri yolunu (common bus) kapsamaktadır. Tasarım, modüller bir yaklaşımla beş temel bileşenden oluşmaktadır: 16-bitlik genel amaçlı yazmaç (reg16), 12-bitlik yazmaç (reg12), aritmetik-mantıksal birim (ALU), senkron bellek (ram4096) ve 8 kaynaklı ortak veri yolu (common_bus). Her bileşen bağımsız bir Verilog modül olarak tanımlanmış ve üst modül (mano_computer) içerisinde yapısal (structural) mimari ile birleştirilmiştir (PÇ1)

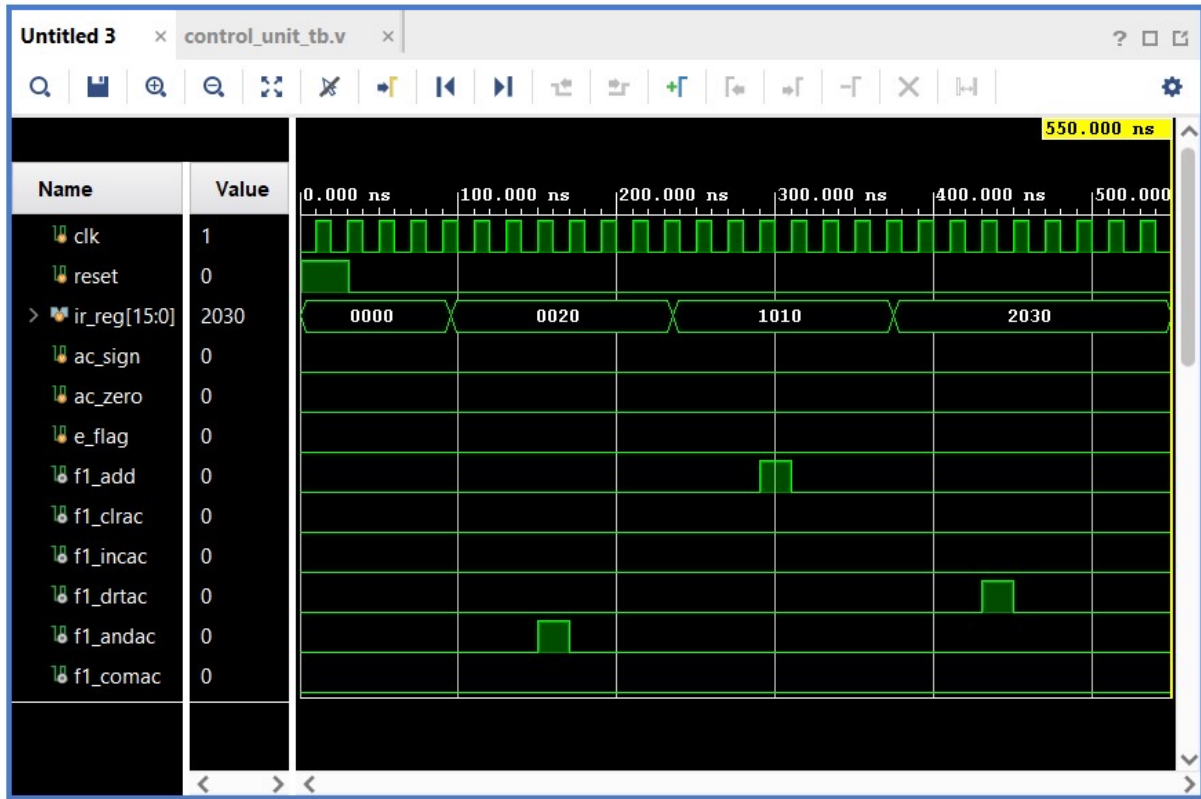
4.1.1. Yazmaç Modülleri

Sistemde iki farklı genişlikte yazmaç modülü kullanılmaktadır 16-bitlik reg16 modülü AC (Accumulator), DR (Data Register), TR (Temporary Register) ve IR (Instruction Register) yazmaçları için; 12-bitlik reg12 modülü ise AR (Address Register) ve PC (Program Counter) yazmaçları için kullanılmaktadır Her iki modül de asenkron reset, senkron sıfırlama(clr), paralel yükleme (load) ve bir arttırma (inc) işlemlerini desteklemektedir. Öncelik sırası clr > load > inc şeklindedir; bu sıralama tasarım belirliliğini garanti altına almaktadır. [1] (PÇ1)

Port	Yön	Genişlik	Açıklama
clk	Giriş	1 bit	Sistem saat sinyali
reset	Giriş	1 bit	Asenkron

			sıfırlama
load	Giriş	1 bit	Paralel yükleme izni
clr	Giriş	1 bit	Senkron sıfırlama
inc	Giriş	1 bit	Bir arttırma
data_in	Giriş	16 bit	Yüklenecek veri
data_out	Çıkış	16 bit	Yazmaç çıkışı

Tablo 4.1. reg16 Modulu Port Listesi (AC, DR, TR, IR)



Şekil 4.2. Vivado Simülasyon Dalga Formu

reg12 modülü reg16 ile aynı arayüze sahiptir; tek fark data_in ve data_out portlarının 12-bit genişliğinde olmasıdır. Bu modül AR ve PC yazmaçları için kullanılmaktadır PC için inc sinyali her fetch döngüsünde PC'yi bir arttırmak üzere; AR için ise load sinyali veri yolundan adres yükleme işleminde aktif hale gelmektedir. (PÇ1)

Port	Yön	Genişlik	Açıklama
clk	Giriş	1 bit	Sistem saat sinyali
reset	Giriş	1 bit	Asenkron sıfırlama
load	Giriş	1 bit	Paralel yükleme izni
clr	Giriş	1 bit	Senkron sıfırlama

inc	Giriş	1 bit	Bir arttırma (PC için)
data_in	Giriş	12 bit	Yüklenecek adres verisi
data_out	Çıkış	12 bit	Yazmaç çıkışı

Tablo 4.3. reg12 Modülü Port Listesi (AR, PC)

4.1.2. Aritmetik ve Mantıksal Birim (ALU)

ALU modülü, AC yazmacı üzerinde 8 farklı işlemi gerçekleştirebilen kombinasyonel bir birimdir. Girişler olarak 16-bitlik AC değeri (a), 16-bitlik DR değeri (b), 3-bitlik işlem kodu (op) ve mevcut taşıma biti (carry_in / E) almaktadır. Çıkış olarak 16-bitlik işlem sonucu (result) ve yeni taşıma biti (carry_out) üretir. ALU, mikroprogramlanmış kontrol biriminin F1 ve F2 alanlarından üretilen işlem kodlarıyla yönetilmektedir (PÇ1) [1]

Port	Yön	Genişlik	Açıklama
a	Giriş	16 bit	AC yazmaç değeri
b	Giriş	16 bit	DR yazmaç değeri
op	Giriş	3 bit	İşlem seçim kodu
carry_in	Giriş	1 bit	Mevcut E (taşıma) biti
result	Çıkış	16 bit	İşlem sonucu
carry_out	Çıkış	1 bit	Yeni E (taşıma) biti

Tablo 4.4. ALU Modülü Port Listesi

Op Kodu	İşlem	Açıklama
000	PASSA	result <- a (veri yolu geçişi)
001	ADD	result <- a + b, E <- carry
010	AND	result <- a AND b
011	COM	result <- NOT a (1'e tümleyen)
100	SHR	result <- E & a[15:1], E <- a[0]
101	SHL	result <- a[14:0] & E, E <- a[15]
110	INC	result <- a + 1 (bir arttır)
111	OR	result <- a OR b

Tablo 4.5. ALU İşlem Kodları ve İşlevleri

ADD ve INC işlemlerinde 17-bitlik ara toplam kullanılarak taşıma biti (carry_out) doğal olarak hesaplanmaktadır. SHR işleminde mevcut E biti MSB konumuna yerleştirilir ve LSB yeni E değeri olarak çıkarılır. SHL işleminde ise E biti LSB'ye yerleştirilir ve MSB yeni E değeri olarak çıkarılır. Bu

dairesel kaydırma mekanizması Mano bilgisayarının CIR ve CIL komutlarını doğru biçimde desteklemektedir. (PÇ1)

4.1.3. Ana Bellek (RAM)

RAM modülü 4096 x 16-bitlik senkron bir bellek olarak tasarlanmıştır. Saat sinyalinin yükselen kenarında read veya write sinyaline göre okuma ya da yazma işlemi gerçekleştirilir. Aynı çevrimde hem read hem write aktif olduğunda yazma önceliklidir. Adres girişi 12-bit genişliğinde olup AR (Address Register) tarafından sağlanmaktadır; veri girişi ve çıkışı 16-bit genişliğindedir. FPGA sentezinde bu modül BRAM (Block RAM) kaynaklarına eşlenmektedir. (PÇ1) [1]

Port	Yon	Genislik	Aciklama
clk	Giriş	1 bit	Sistem saat sinyali
address	Giriş	12 bit	Bellek adresi (AR'den)
data_in	Giriş	16 bit	Yazılacak veri (DR'den)
read	Giriş	1 bit	Okuma izni
write	Giriş	1 bit	Yazma izni
data_out	Çıkış	16 bit	Okunan veri

Tablo 4.6. ram4096 Modülü Port Listesi

4.1.4. Ortak Veri Yolu (Common Bus)

Ortak veri yolu, 3-bitlik seçim sinyali (sel) ile 8 farklı kaynaktan birini 16-bitlik veri yoluna bağlayan bir çoklayıcı (multiplexer) yapısındadır. 12-bitlik yazmaçlar (AR, PC) veri yoluna bağlanırken üst 4 bit sıfır ile doldurulur; 8-bitlik INPR giriş yazmaçları için ise üst 8 bit sıfır ile doldurulmaktadır. Seçim sinyalleri mikroprogramlanmış kontrol biriminin F3 alanı tarafından üretilir ve üst modül (mano_computer) içerisinde kontrol sinyallerine göre atanmaktadır. (PÇ1)

Port	Yön	Genişlik	Açıklama
sel	Giriş	3 bit	Kaynak seçim sinyali
ar_in	Giriş	12 bit	AR yazmac girişi
pc_in	Giriş	12 bit	PC yazmac girişi
dr_in	Giriş	16 bit	DR yazmac girişi
ac_in	Giriş	16 bit	AC yazmac girişi
ir_in	Giriş	16 bit	IR yazmac girişi
tr_in	Giriş	16 bit	TR yazmac girişi
inpr_in	Giriş	8 bit	Giris yazmacı (INPR)
bus_out	Çıkış	16 bit	Ortak veri yolu çıkışı

Tablo 4.7. common_bus Modülü Port Listesi

Seçim (sel)	Kaynak	Açıklama
000	(yok / HiZ)	Hiçbir kaynak seçilmemiş, bus = 0
001	AR (12-bit)	Üst 4 bit sıfır ile doldurulur
010	PC (12-bit)	Üst 4 bit sıfır ile doldurulur
011	DR (16-bit)	Tam 16-bit veri aktarımı
100	AC (16-bit)	Tam 16-bit veri aktarımı
101	IR (16-bit)	Tam 16-bit veri aktarımı
110	TR (16-bit)	Tam 16-bit veri aktarımı
111	INPR (8-bit)	Üst 8 bit sıfır ile doldurulur

Tablo 4.8. Ortak Veri Yolu Kaynak Secim Tablosu

4.1.5. Tasarım Kararları

Veri yolu tasarımında alınan temel kararlar şunlardır; (i) Tüm yazmaçlar senkron tasarım prensibiyle gerçekleştirilmiş olup asenkron reset desteği yalnızca sistem başlatma için kullanılmaktadır; bu yaklaşım simülasyonda güvenilir zamanlama sağlamaktadır. (ii) RAM modülü tek portlu senkron bellek olarak tasarlanmıştır; aynı çevrimde okuma ve yazma çatışmasını önlemek için yazma işlemine öncelik verilmektedir. (iii) Ortak veri yolu 16-bit genişliğinde seçilmiş olup dar yazmaçlar (AR: 12-bit, INPR: 8-bit) sıfır uzatma (zero-extension) ile veri yoluna bağlanmaktadır. (iv) ALU tamamen kombinasyonel olarak tasarlanmıştır sonuçların yazmaçlara yansıtılması saat kenarında kontrol sinyalleriyle sağlanmaktadır. (v) Yazmaç modüllerinde öncelik sırası clr > load > inc olarak belirlenerek tasarım belirliliğini garanti altına almaktadır. (PÇ1, PÇ5)

4.2. Mikrokomut Formatı

F1 alanı ağırlıklı olarak ALU ve akümülatör (AC) işlemlerini, F2 alanı ikincil ALU işlemleri ve yazmaç transferlerini, F3 alanı ise bellek erişimi ve giriş/çıkış işlemlerini kodlamaktadır. Bu üçlü ayırım, Mano bilgisayarının mikro-işlem gereksinimlerinin sistematik biçimde sınıflandırılmasıyla elde edilmiştir. Her alan 3 bit genişliğinde olduğundan, NOP dahil 8 farklı mikro-işlem tanımlanabilmektedir. Kodlama sırasında aynı donanım kaynağına erişen mikro-işlemler aynı alana yerleştirilerek olası çakışmalar önlenmiştir. (PÇ1, PÇ5) Bu tasarımda benimsenen 20 bitlik mikrokomut formatı Bölüm 3.3'te tanıtılmıştır. F1, F2 ve F3 alanlarının her biri 3 bit olup toplamda 7'ser farklı mikro-işlem kodlanabilmektedir. Bir mikrokomut içinde aynı anda F1, F2 ve F3 alanlarının tamamı bağımsız olarak aktif olabilir; bu yapı yatay mikrokomut (horizontal microinstruction) olarak sınıflandırılır

ve bir çevrimde birden fazla mikro-işlemin paralel yürütülmesine olanak tanır. (PÇ1)

Tablo 4.9 – F1 Alanı Kodlaması (ALU / AC İşlemleri):

Kod	Sembol	Mikro-işlem	RTL Gösterimi
000	NOP	İşlem yok	-
001	ADD	Toplama	$AC \leftarrow AC + DR, E \leftarrow Cout$
010	CLRAC	AC sıfırlama	$AC \leftarrow 0$
011	INCAC	AC artırma	$AC \leftarrow AC + 1$
100	DRTAC	DR'den AC transfer	$AC \leftarrow DR$
101	ANDAC	Mantıksal AND	$AC \leftarrow AC \wedge DR$
110	COMAC	AC tümleyeni	$AC \leftarrow AC'$
111	CLRE	E sıfırlama	$E \leftarrow 0$

Tablo 4.10 – F2 Alanı Kodlaması (Yazmaç Transfer / Kaydırma):

Kod	Sembol	Mikro-işlem	RTL Gösterimi
000	NOP	İşlem yok	-
001	SUB	Çıkarma	$AC \leftarrow AC - DR$
010	OR	Mantıksal OR	$AC \leftarrow AC \vee DR$
011	SHL	Sola Kaydırma	$AC \leftarrow shl\ AC, AC(0) \leftarrow E, E \leftarrow AC(15)$
100	SHR	Sağa Kaydırma	$AC \leftarrow shr\ AC, AC(15) \leftarrow E, E \leftarrow AC(0)$
101	INCPC	PC artırma	$PC \leftarrow PC + 1$
110	ARTPC	AR'den PC'ye transfer	$PC \leftarrow AR$
111	COME	E tümleyeni	$E \leftarrow E'$

Tablo 4.11 – F3 Alanı Kodlaması (Bellek / G-Ç) :

Kod	Sembol	Mikro-işlem	RTL Gösterimi
000	NOP	İşlem yok	-
001	READ	Bellekten Okuma	$DR \leftarrow M[AR]$
010	WRITE	Belleğe Yazma	$M[AR] \leftarrow DR$
011	PCTAR	PC'den AR'ye transfer	$AR \leftarrow PC$
100	IRTAR	IR'den AR'ye transfer	$AR \leftarrow IR(11:0)$
101	ACTDR	AC'den DR'ye transfer	$DR \leftarrow AC$
110	INCDR	DR artırma	$DR \leftarrow DR + 1$
111	DRTIR	DR'den IR'ye transfer	$IR \leftarrow DR$

Tablo 4.12 – CD Alanı (Dallanma Koşulu) :

Kod	Sembol	Koşul
00	U	Koşulsuz (her zaman doğru)
01	I	IR(15) – dolaylı adresleme biti
10	S	AC(15) – işaret biti
11	Z	AC = 0 – sıfır bayrağı

Tablo 4.13 – BR Alanı (Dallanma Tipi) :

Kod	Sembol	İşlem
00	JMP	$CAR \leftarrow AD$ (koşullu atlama)
01	CALL	$SBR \leftarrow CAR+1$, $CAR \leftarrow AD$ (alt program)
10	RET	$CAR \leftarrow SBR$ (alt programdan dönüş)
11	MAP	$CAR \leftarrow '0' \& IR(14:12) \& "000"$ (eşleme)

Dallanma mekanizmasında CD alanı koşulu değerlendirir; koşul sağlanırsa BR alanına göre dallanma gerçekleşir, sağlanmazsa CAR sıralı olarak bir artırılır ($CAR \leftarrow CAR + 1$). MAP işlemi, IR yazmacındaki komut kodunu (opcode) kontrol belleği adresine dönüştürerek her makine komutunun ilgili mikroprogram bloğuna yönlendirilmesini sağlamaktadır. Bu eşleme fonksiyonu ile 3 bitlik opcode, 8'er konumluk bloklara (0x00, 0x08, 0x10, ...) atanmaktadır. (PÇ1)

4.3. Kontrol Belleği ve Mikroprogram

Kontrol belleği, Verilog'da bir dizi (array) olarak tanımlanmıştır. Fetch döngüsü 0x40 adresinden başlamakta olup dört mikrokomut adımından oluşmaktadır: (1) $AR \leftarrow PC$, (2) $DR \leftarrow M[AR]$ ile eş zamanlı $PC \leftarrow PC+1$, (3) $IR \leftarrow DR$, (4) $AR \leftarrow IR(11:0)$ ve MAP işlemiyle $CAR \leftarrow '0' \& IR[14:12] \& '000'$. Dördüncü adımda MAP işlemi, ilgili komutun mikroprogram bloğuna yönlendirmeyi sağlar. (PÇ5)

Kontrol belleği 128 × 20 bit boyutunda olup Verilog'da dizi (array) olarak tanımlanmıştır. Kontrol belleğinin adres haritası Tablo 4.13'te sunulmuştur. (PÇ5)

Tablo 4.14 – Kontrol Belleği Adres Haritası:

Adres Aralığı	Komut	Opcode
0x00 - 0x07	AND	000
0x08 - 0x0F	ADD	001
0x10 - 0x17	LDA	010
0x18 - 0x1F	STA	011
0x20 - 0x27	BUN	100
0x28 - 0x2F	BSA	101
0x30 - 0x37	ISZ	110
0x38 - 0x3F	REG-REF/IO	111
0x40 - 0x47	FETCH döngüsü	-
0x48 - 0x4F	INDIRECT	-
0x50 - 0x7F	Boş (rezerv)	-

MAP fonksiyonu $CAR \leftarrow '0' \& IR(14:12) \& '000'$ biçiminde çalıştığından, her opcode değeri 8 ardışık mikrokomut konumuna sahip bir bloğa eşlenmektedir. Bu yapı, her komutun en fazla 8 mikrokomut adımıyla yürütülmesine olanak tanımaktadır. Fetch döngüsü 0x40 adresinden başlamakta ve sistem reset sonrasında CAR bu adrese yüklenmektedir. (PÇ5)

Tablo 4.15 – Fetch Döngüsü Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x40	NOP	NOP	PCTAR	U	JMP	0x41	AR $\leftarrow$ PC
0x41	NOP	INCPC	READ	U	JMP	0x42	DR $\leftarrow$ M[AR], PC $\leftarrow$ PC + 1
0x42	NOP	NOP	DRTIR	U	JMP	0x43	IR $\leftarrow$ DR
0x43	NOP	NOP	IRTAR	U	MAP	0x00	AR $\leftarrow$ IR(11:0), MAP decode

Fetch döngüsü dört adımdan oluşmaktadır. İlk adımda program sayacının (PC) değeri adres yazmacına (AR) aktarılır. İkinci adımda bellekten komut okunarak veri yazmacına (DR) yüklenir ve eş zamanlı olarak PC bir artırılır. Üçüncü adımda DR'deki komut kodu komut yazmacına (IR) aktarılır. Dördüncü adımda IR'nin alt 12 biti AR'ye yüklenir (operand adresi) ve MAP işlemi ile CAR, ilgili komutun mikroprogram bloğuna yönlendirilir. (PÇ5)

Her komut bloğunun ilk mikrokod satırı dolaylı adresleme kontrolü yapmaktadır. CD=I (IR[15]) koşuluyla CALL INDIRECT gerçekleştirilir; dolaylı bit aktif değilse CAR sıralı olarak bir sonraki adrese ilerler. INDIRECT alt programı 0x48 adresindedir ve bellekten okunan dolaylı adresi AR'ye yükledikten sonra RET ile çağrılan yere geri döner. (PÇ1)

Tablo 4.16 – AND Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x00	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x01	NOP	NOP	READ	U	JMP	0x02	DR $\leftarrow$ M[AR]
0x02	ANDAC	NOP	NOP	U	JMP	0x40	AC $\leftarrow$ AC $\wedge$ DR, $\rightarrow$ FETCH

Tablo 4.17 – ADD Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x08	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x09	NOP	NOP	READ	U	JMP	0x0A	DR $\leftarrow$ M[AR]
0x0A	ADD	NOP	NOP	U	JMP	0x40	AC $\leftarrow$ AC + DR, E

							← Cout, → FETCH
--	--	--	--	--	--	--	--------------------

Tablo 4.18 – LDA Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x10	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x11	NOP	NOP	READ	U	JMP	0x12	DR ← M[AR]
0x12	DRTAC	NOP	NOP	U	JMP	0x40	AC ← DR, → FETCH

Tablo 4.19 – STA Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x18	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x19	NOP	NOP	ACTDR	U	JMP	0x1A	DR ← AC
0x1A	NOP	NOP	WRITE	U	JMP	0x40	M[AR] ← DR, → FETCH

Tablo 4.20 – BUN Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x20	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x21	NOP	ARTPC	NOP	U	JMP	0x40	PC ← AR, → FETCH

Tablo 4.21 – BSA Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x28	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL INDIRECT
0x29	NOP	NOP	ACTDR	U	JMP	0x2A	DR ← PC (dönüş adresi)
0x2A	NOP	NOP	WRITE	U	JMP	0x2B	M[AR] ← DR
0x2B	NOP	ARTPC	NOP	U	JMP	0x2C	PC ← AR
0x2C	NOP	INCPC	NOP	U	JMP	0x40	PC ← PC + 1, → FETCH

Tablo 4.22 – ISZ Komutu Mikroprogram Tablosu:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x30	NOP	NOP	NOP	I	CALL	0x48	if I=1: CALL

							INDIRECT
0x31	NOP	NOP	READ	U	JMP	0x32	DR ← M[AR]
0x32	NOP	NOP	INCDR	U	JMP	0x33	DR ← DR + 1
0x33	NOP	NOP	WRITE	U	JMP	0x34	M[AR] ← DR
0x34	DRTAC	NOP	NOP	U	JMP	0x35	AC ← DR (Z koşulu için köprü adım)
0x35	NOP	INCPC	NOP	Z	JMP	0x40	if AC=0 (DR=0 ile eşdeğer): PC++, → FETCH
0x36	NOP	NOP	NOP	U	JMP	0x40	→ FETCH (DR≠0 durumu)

Tasarım notu: ISZ komutu, artırılmış DR değerinin sıfır olup olmadığını kontrol etmeyi gerektirir; ancak bu mimaride CD alanında doğrudan DR = 0 koşulu için bir kodlama yer almamaktadır. Bu kısıt, 0x34 adresinde bir köprü adım (DRTAC: AC ← DR) ile çözülmüş; ardından 0x35 satırında Z koşulu (AC = 0) değerlendirilerek beklenen 'if DR=0 → PC++' davranışı elde edilmiştir. Köprü adımın bedeli, ISZ komutunun yürütülmesi süresince AC içeriğinin geçici olarak değişmesidir; bu durum Mano referansındaki ISZ semantiğine kıyasla AC'nin bozulması anlamına gelir. Üretim ortamında bu kısıtın giderilmesi için CD alanına '11' yerine ayrı bir kodla DR-zero koşulu eklenmesi veya CD genişliğinin 3 bite çıkarılması uygun bir iyileştirme olacaktır. (PÇ5)

Tablo 4.23 – INDIRECT Alt Programı:

Adres	F1	F2	F3	CD	BR	AD	Açıklama
0x48	NOP	NOP	READ	U	JMP	0x49	DR ← M[AR]
0x49	NOP	NOP	DRTIR	U	JMP	0x4A	IR ← DR (geçici)
0x4A	NOP	NOP	IRTAR	U	RET	0x00	AR ← IR(11:0)

Dolaylı adresleme alt programı üç mikrokmut adımından oluşmaktadır. Önce AR'nin gösterdiği bellek konumundan dolaylı adres okunarak DR'ye yüklenir. Ardından DR'deki değer geçici olarak IR'ye aktarılır ve son adımda IR'nin alt 12 biti AR'ye yüklenerek gerçek operand adresi elde edilir. RET işlemi

ile CAR, CALL öncesinde SBR'ye kaydedilmiş olan dönüş adresine yüklenmektedir. (PÇ1, PÇ5)

4.4. Performans Kriterleri ve Sınırlamalar

Tasarlanan mikroprogramlanmış kontrol birimi, her makine komutunu 6-12 saat darbesi (clock cycle) içinde tamamlamaktadır. Fetch aşaması 4 çevrim, execute aşaması komuta ve adresleme moduna bağlı olarak 2-8 çevrim sürmektedir. Hardwired kontrole kıyasla komut başına daha fazla çevrim gerekmesi, mikroprogramlanmış mimarinin bilinen bir hız dezavantajını oluşturmaktadır. (PÇ5) [1]

Tasarımın temel kısıtları şöyle sıralanabilir: (i) 7 bitlik CAR ile kontrol belleği 128 konumla sınırlıdır; bu durum, genişletilmiş komut setlerinde mikroprogram alanının dikkatli yönetilmesini zorunlu kılmaktadır. (ii) Dolaylı adresleme subroutine'i ek çevrim gerektirdiğinden performansı olumsuz etkileyebilir. (PÇ5)

Tablo 4.23'te her komut kategorisi için gereken toplam saat çevrimi sayısı özetlenmiştir. Çevrim sayılarına fetch döngüsünün 4 çevrimi dahildir. (PÇ5)

Tablo 4.24 – Komut Başına Saat Çevrimi Analizi:

Komut Tipi	Fetch	Dolaylı	Execute	Toplam (Doğrudan)	Toplam (Dolaylı)
AND, ADD, LDA	4	3	3	7	10
STA	4	3	3	7	10
BUN	4	3	2	6	9
BSA	4	3	5	9	12
ISZ	4	3	7	11	14
Register-ref	4	-	2	6	-

Tablodan görüleceği üzere, en hızlı komutlar BUN ve register-reference komutları (6 çevrim), en yavaş komut ise dolaylı adreslemeli ISZ komutudur

(14 çevrim – Z koşulu için ek AC-DR köprü adımı dahil). Hardwired kontrol devresinde aynı komutlar 3-5 çevrimde tamamlanmaktayken, mikroprogramlanmış yapıda bu süre yaklaşık iki kat artmaktadır. Bu fark, her çevrimde kontrol belleğinden bir mikrokod okunması gerekliliğinden kaynaklanmaktadır. Buna karşın mikroprogramlanmış yapının sağladığı esneklik ve tasarım basitliği, eğitim amaçlı ve orta ölçekli sistemlerde bu performans farkını kabul edilebilir kılmaktadır. (PÇ5)

5. UYGULAMA VE SİMÜLASYON

5.1. Verilog İmplementasyonu

Sistemin tamamı Verilog donanım tanımlama dili ile geliştirilmiştir. Tasarım ve simülasyon aşamalarında Xilinx Vivado 2025.1 ortamı kullanılmış; tüm modüllerde IEEE standart wire ve reg veri tipleri tercih edilmiştir. Tasarım toplam 11 Verilog modülünden oluşmakta olup üç katmanlı hiyerarşik bir mimari izlenmektedir. Her modül bağımsız bir .v dosyası olarak tanımlanmış ve üst modül (mano_computer) içerisinde yapısal (structural) örnekleme ile birleştirilmiştir. [4][5] (PÇ5)

5.1.1. Modül Hiyerarşisi

Birinci katman olan veri yolu katmanı, temel veri depolama ve işlem bileşenlerini içerir: reg16 (16-bit yazmaç, 4 örnekleme: AC, DR, TR, IR), reg12 (12-bit yazmaç, 2 örnekleme: AR, PC), alu (8 işlemli aritmetik-mantıksal birim), ram4096 (4096 x 16-bit senkron bellek) ve common_bus (8 kaynaklı 16-bit ortak veri yolu). INPR (8-bit giriş) ve OUPR (8-bit çıkış) yazmaçları bu sürümde ayrı bir modül olarak değil mano_computer üst modülünün portları/i-c kayıtları olarak gerçekleştirilmiştir; INPR doğrudan ortak veri yoluna bağlanmakta, OUPR ise üst modülde 8-bit reg olarak tanımlanmaktadır .Bu katmandaki bileşenler kontrol biriminden bağımsız olarak test edilebilir yapıdadır. (PÇ5)

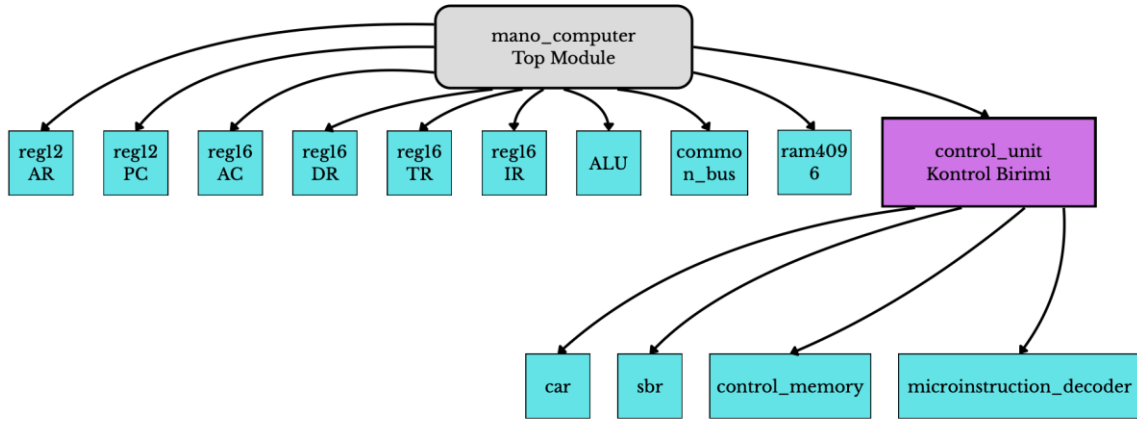
İkinci katman olan kontrol birimi katmanı, mikroprogramlanmış kontrol mimarisinin çekirdek bileşenlerini barındırır: car (Control Address Register, 7-bit), sbr (Subroutine Register, 7-bit), control_memory (128 x 20-bit ROM) ve microinstruction_decoder (20-bit mikrokodu 21 ayrı kontrol sinyaline çeviren kombinasyonel çözücü). Bu bileşenler, kontrol birimi üst modülü (control_unit) içerisinde birleştirilmiştir. control_unit modülü ayrıca koşul değerlendirme (CD alanı) ve dallanma mantığı (BR alanı) için gerekli kombinasyonel lojiği de barındırmaktadır. (PÇ5)

Üçüncü katman, mano_computer üst modülüdür. Bu modül veri yolu katmanı ile kontrol birimi katmanını yapısal (structural) mimari kullanarak birleştirir. Kontrol biriminden gelen F1, F2, F3 sinyalleri, üst modül içerisinde yazmaç yükleme, ALU işlem kodu seçimi, bus kaynaklı seçimi ve E biti güncelleme

mantigina donusturulmektedir. Sistemin dis arayuzu saat (clk), sifirlama (reset), 8-bit giris yazmaci (inpr), 8-bit cikis yazmaci (outr) ve debug portlarindan olusmaktadir. (PÇ5) (PÇ

Modül	Dosya	Katman	İşlev
reg16	reg16.v	Veri Yolu	16-bit yazmaç (AC, DR, TR, IR)
reg12	reg12.v	Veri Yolu	12-bit yazmaç (AR, PC)
alu	alu.v	Veri Yolu	8 işlemlili ALU birimi
ram4096	ram4096.v	Veri Yolu	4096x16 senkron bellek
common_bus	common_bus.v	Veri Yolu	8 kaynaklı ortak veri yolu
car	car.v	Kontrol Birimi	7-bit kontrol adres yazmaci
sbr	sbr.v	Kontrol Birimi	7-bit alt program yazmaci
control_memory	control_memory.v	Kontrol Birimi	128x20 bit kontrol belleği (ROM)
micro_decoder	microinstruction_decoder.v	Kontrol Birimi	20-bit mikrokod çözücü
control_unit	control_unit.v	Kontrol Birimi	Kontrol birimi üst modülü
mano_computer	mano_computer.v	Üst Modül	Sistem üst modülü

Tablo 5.1 – Sistem Bileşenleri ve Modül Hiyerarşisi



Şekil 5.1. Verilog Modül Hiyerarşi Diyagramı (Sistem Üst Modülü ve Alt Modüller)

5.1.2. Veri Yolu Katmanı Detayları

Yazmaç modülleri (reg16 ve reg12), saat sinyalinin yükselen kenarında (posedge clk) çalışan senkron yapılarıdır. Her iki modül de asenkron sıfırlama (posedge reset), senkron sıfırlama (clr), paralel yükleme (load) ve bir arttırma (inc) işlemlerini desteklemektedir. Öncelik sırası reset > clr > load > inc olarak belirlenmiş olup bu sıralama Verilog kodunda if-else if zincirleme yapısındaki koşul sıralamasıyla garanti altına alınmıştır. reg16 modülü AC, DR, TR ve IR yazmaçları için, reg12 modülü AR ve PC yazmaçları için örneklenmiştir. INPR (8-bit giriş) doğrudan üst modül portu olarak veri yoluna bağlanmış, OUTR (8-bit çıkış) ise üst modülde 8-bit reg olarak tutulmaktadır. Bu yapı, yazmaçların doğrudan mano_computer içinde değil de hiyerarşiye uygun biçimde modüller halinde tanımlanmasını sağlamaktadır. (PÇ1) (PÇ13)

ALU modülü tamamen kombinasyonel (always @(*)) bir birim olarak tasarlanmıştır. 16-bit genişliğindeki iki giriş (a: AC değeri, b: DR değeri), 3-bit işlem kodu (op) ve mevcut tasima biti (carry_in) ile çalışmaktadır. ADD ve INC işlemlerinde 17-bit ara toplam ({1'b0, a} + {1'b0, b}) kullanılarak carry_out doğal olarak hesaplanmaktadır SHR işleminde carry_in MSB konumuna yerleştirilirken a[0] yeni carry_out olur; SHL işleminde ise carry_in LSB'ye yerleştirilirken a[15] yeni carry_out olur. Bu dairesel kaydırma mekanizması Mano bilgisayarının CIR ve CIL komutlarını doğru biçimde desteklemektedir. (PÇ1)

RAM modülü (ram4096) 4096 x 16-bit kapasiteli senkron bir bellek olarak gerçekleştirilmiştir. Verilog'da 'reg [15:0] mem [0:4095]' dizisi ile tanımlanmış olup initial bloğu ile tüm konumlar sıfıra alınmaktadır. Saat sinyalinin yükselen kenarında write sinyali öncelikli olacak şekilde okuma/yazma işlemi gerçekleştirilir. Bellekten yapılan senkron okuma (synchronous read) işleminde verinin DR yazmacına ulaşması bir sonraki saat darbesinde (1 cycle

delay) gerçekleştiğinden, mikroprogramdaki FETCH aşamasının zamanlaması (timing) bu gecikme gözetilerek tasarlanmıştır. FPGA sentezinde bu modül otomatik olarak Block RAM (BRAM) kaynaklarına eslenmektedir. (PÇ1)

Ortak veri yolu (common_bus), 3-bitlik seçim sinyali (sel) ile 8 farklı kaynaktan birini 16-bitlik veri yoluna bağlayan kombinasyonel bir çoklayıcı (multiplexer) yapısıdır. 12-bit genişliğindeki yazmaclar (AR, PC) veri yoluna bağlanırken üst 4 bit sıfır ile doldurulur ({4'b0000, ar_in}); 8-bit genişliğindeki INPR yazmacı için üst 8 bit sıfır ile doldurulur ({8'h00, inpr_in}). Seçim sinyali 000 olduğunda veri yolu sıfır değerini taşır ve hiçbir yazma işlemi gerçekleşmez. (PÇ1)

5.1.3. Kontrol Birimi Katmanı Detayları

CAR (Control Address Register) modülü, okunacak bir sonraki mikrokomutun adresini tutan 7-bitlik bir yazmacıdır. Reset durumunda CAR, FETCH döngüsünün başlangıç adresi olan 0x40 (7'b1000000) değerine yüklenir; bu sayede sistem her açıldığında otomatik olarak komut çekme döngüsüne baslar. Normal işleyişte CAR ya load sinyali ile yeni bir adres yükler (dallanma durumları) ya da inc sinyali ile bir artar (sıralı işleyiş). (PÇ1)

SBR (Subroutine Register) modülü, alt program çağrısı (CALL) anında dönüş adresinin saklandığı 7-bitlik bir yazmacıdır. CALL işlemi sırasında SBR'a CAR + 1 değeri yüklenirken, RET işlemi sırasında SBR'daki değer CAR'a geri yüklenerek çağrının yapıldığı noktaya dönüş sağlanır. Bu mekanizma özellikle dolaylı adresleme alt programı (INDIRECT subroutine, 0x48) için kullanılmaktadır. (PÇ1)

Kontrol belleği (control_memory), 128 x 20-bit kapasiteli bir ROM olarak tasarlanmıştır. Verilog'da 'reg [19:0] rom [0:127]' dizisi ile tanımlanmış olup tüm mikroprogram içeriği initial bloğu içerisinde localparam sabitleri kullanılarak okunaklı biçimde kodlanmıştır. Her mikrokomut {F1, F2, F3, CD, BR, AD} şeklinde birleştirilmiştir. Bellek haritası şu şekilde düzenlenmiştir: AND komutu 0x00-0x02, ADD komutu 0x08-0x0A, LDA komutu 0x10-0x12, STA komutu 0x18-0x1A, BUN komutu 0x20-0x21, BSA komutu 0x28-0x2C, ISZ komutu 0x30-0x36, Register-ref/IO 0x38-0x3F, FETCH döngüsü 0x40-0x43 ve INDIRECT alt programı 0x48-0x4A adreslerinde konumlandırılmıştır. Kontrol belleğine erişim kombinasyonel assign deyimi (assign data = rom[address]) ile sağlanmakta olup bellek okuma gecikmesi yalnızca kablo gecikmesine (wire delay) bağlıdır. (PÇ5)

Mikrokomut çözücü (microinstruction_decoder), 20-bitlik mikrokomutu 21 ayrı kontrol sinyaline dönüştüren tamamen kombinasyonel bir modüldür. Giriş olarak aldığı 20-bit mikrokomutun üst 9 bitini (F1[19:17], F2[16:14], F3[13:11]) 3'er bitlik alanlara ayırır ve her alan için 7 ayrı eşitlik karşılaştırılması (assign f1_add = (f1 == 3'b001)) uygulayarak ilgili kontrol sinyalini üretir. Kalan 6 bit (CD[10:9], BR[8:7], AD[6:0]) doğrudan çıkış portlarına atanır. Bu yaklaşım her saat darbesinde kontrol belleğinden okunan mikrokomutun anında ilgili kontrol sinyallerine dönüştürülmesini sağlamaktadır. (PÇ5)

5.1.4. Kontrol Birimi Dallanma Mantiğı

Kontrol birimi üst modülü (control_unit.v), CAR, SBR, kontrol belleği ve mikrokomut çözücüyü birleştirmenin yani sıra iki kritik kombinasyonel mantık bloğu içerir: koşul değerlendirme ve dallanma yönetimi. (PÇ5)

Koşul değerlendirme, CD alanının 2-bitlik değerine göre çalışan bir case ifadesiyle gerçekleştirilir. CD = 00 (U: Unconditional) durumunda koşul her zaman doğrudur (condition = 1). CD = 01 (I) durumunda IR[15] dolaylı adresleme biti kontrol edilir. CD = 10 (S) durumunda AC'nin işaret biti (ac_sign = AC[15]) değerlendirilir. CD = 11 (Z) durumunda AC'nin sıfır olup olmadığı (ac_zero) kontrol edilir. (PÇ1)

Dallanma yönetimi, BR alanının 2-bitlik değerine göre dört farklı işlem üretir. BR = 00 (JMP) durumunda koşul sağlanıyor ise CAR'a AD alanı yüklenir, sağlanmıyor ise CAR bir arttırılır (siralı isleyiş). BR = 01 (CALL) durumunda koşul sağlandığında SBR'a CAR + 1 değeri kaydedilir ve CAR'a AD yüklenir; bu mekanizma dolaylı adresleme alt programı çağırısı için kullanılır. BR = 10 (RET) durumunda koşul değerlendirilmeksizin CAR'a SBR değeri yüklenerek alt programdan dönüş sağlanır. BR = 11 (MAP) durumunda CAR'a {1'b0, IR[14:12], 3'b000} formülü ile hesaplanan adres yüklenir; bu formül her komutun opcode'unu kontrol belleğindeki 8 mikrokomutluk yürütme bloğuna otomatik olarak yönlendirir. (PÇ5)

5.1.5. Üst Modül Entegrasyonu

Üst modül (mano_computer.v) içerisinde kontrol sinyallerinin veri yolu bileşenlerine bağlanması şu ilkelerle gerçekleştirilmiştir. AR yazmacı, f3_pctar (AR <- PC) veya f3_irtar (AR <- IR[11:0]) sinyalleriyle yüklenirken bus secimi otomatik olarak ilgili kaynağa yönlendirilir: f3_pctar aktif ise bus_sel = 010 (PC), f3_irtar aktif ise bus_sel = 101 (IR), f3_drtir aktif ise bus_sel = 011 (DR), f3_actdr aktif ise bus_sel = 010 (PC) olarak seçilir. ACTDR işleminin bu sürümde şematiği BSA komutu için özelleştirilmiştir: F3 = ACTDR kodlandığında üst modül bus_sel'i PC'ye yönlendirerek DR'ye PC değerinin yüklenmesini sağlar. Bu tasarım kararının gerekçesi, BSA komutunun dönüş adresini bellekte saklamak için PC->DR transferine ihtiyaç duyması ve mevcut F3 kodlamasında PCTDR için ayrı bir konum bulunmamasıdır. Bu yaklaşımın tasarımında gözütılması gereken kısıtlama, aynı F3 kodunun klasik 'AC->DR' transferi (STA komutunda kullanılan) için de paylaşılmasıdır; STA komutunun tam işlevselliği için ya F3 alanına ayrı bir ACTDR_AC kodu eklenmeli ya da bus_sel mantığı komut bağlamına göre koşullandırılmalıdır (PÇ5) (PÇ13)

AC yazmacı, herhangi bir F1 işlem sinyali (f1_add, f1_drtac, f1_andac, f1_comac) veya F2 sinyali (f2_sub, f2_or, f2_shl, f2_shr) aktif olduğunda ALU sonucunu (alu_result) yüklemektedir. f1_clrac sinyali AC'yi senkron olarak sıfırlar, f1_incac sinyali ise AC değerini bir arttırır. DR yazmacı f3_read (bellekten okuma) veya f3_actdr (BSA için PC değerinin yüklenmesi) sinyalleri ile yüklenirken, f3_incdr sinyali DR değerini bir arttırır (ISZ komutu için). PC yazmacı f2_artpc sinyali ile AR değerini yükler (BUN, BSA komutları), f2_inpc ile bir arttırılır (fetch döngüsünde). (PÇ5)

ALU işlem kodu secimi, kontrol biriminin F1 ve F2 alanlarına göre öncelikli koşul ifadeleriyle (if-else if zinciri) belirlenmiştir: f1_add -> 001 (ADD), f1_andac -> 010 (AND), f1_comac -> 011 (COM), f2_shr -> 100 (SHR), f2_shl -> 101 (SHL), f1_incac -> 110 (INC), f2_or -> 111 (OR). Hiçbir kontrol sinyali aktif değilse ALU varsayılan olarak 000 (PASSA) işlemini gerçekleştirerek AC değerini değiştirmeden geçirmektedir. (PÇ5)

E (tasima/extend) biti, saat sinyalinin yükselen kenarında çalışan senkron bir flip-flop ile yönetilmektedir. f1_clre sinyali aktif olduğunda E sıfırlanır; f2_come sinyali aktif olduğunda E tumlenenir (~e_flag); f1_add, f1_incac, f2_shl veya f2_shr sinyallerinden herhangi biri aktif olduğunda E, ALU'nun carry_out çıkışına eşitlenir. Bu yaklaşım, kontrol birimi ile veri yolu arasında net bir arayüz tanımlanarak paralel geliştirme sürecini mümkün kılmıştır. (PÇ5)

Kontrol Sinyali	Verilog İfadesi	İşlev
bus_sel	f3 sinyallerine göre (010, 011, 101 vb.)	Ortak veri yolu kaynak secimi (3-bit)
ar_load	f3_pctar f3_irtar	AR yazmacına yukleme izni
ir_load	f3_drtir	IR yazmacına DR içeriğini yükle
dr_load	f3_read f3_actdr	DR'ye bellek veya bus verisi yükle
ac_load	f1_add f1_drtac ...	AC'ye ALU sonucu yükle
ac_clr	f1_clrac	AC'yi senkron sıfırla
pc_load	f2_artpc	PC'ye AR değerini yükle
pc_inc	f2_incp	PC'yi bir arttır

Tablo 5.2 – Veri Yolu Kontrol Sinyalleri ve Verilog Eşleşmeleri

5.1.6. Tasarım Kararları ve Gerekçeleri

Verilog implementasyonunda alınan temel tasarım kararları şunlardır: (i) Tüm yazmaçlar posedge clk ve posedge reset ile çalışan senkron tasarım prensibiyle gerçekleştirilmiş olup asenkron reset yalnızca sistem başlatma için kullanılmaktadır. (ii) Kontrol belleği Verilog initial bloğu ile başlatılmış olup sentez araçları tarafından otomatik olarak LUT veya BRAM kaynaklarına eslenmektedir. (iii) ALU tamamen kombinasyonel olarak tasarlanmıştır; sonuçların yazmaçlara yansıtılması saat kenarında kontrol sinyalleriyle sağlanmaktadır. (iv) Kontrol birimi ile veri yolu arasındaki arayüz, 21 ayrı tek-bit kontrol sinyali üzerinden tanımlanmıştır; bu yaklaşım iki katmanın

bağımsız olarak geliştirilip test edilmesine olanak tanımıştır (v) BSA komutu için f3_actdr sinyali algılandığında bus secimi PC'ye (010) yönlendirilmek suretiyle DR'ye PC değeri yuklenebilmektedir; bu özel durum kontrol belleğinde ek bir F3 kodu tanımlamaya gerek kalmadan üst modul mantığında çözülmüştür. (PÇ1, PÇ5)

5.2. Test Senaryoları ve Sonuçlar

Tasarımın doğrulanması iki katmanda gerçekleştirilmiştir: (i) kontrol birimi düzeyinde mikrokod/dallanma davranışını sınavan birim testler ve (ii) tüm sistemin tam kod yürütme akışını gözlemleyen entegrasyon testleri. Tüm simülasyonlar Xilinx Vivado 2025.1 ortamında SystemVerilog/IEEE 1364-2005 uyumlu testbench dosyalarıyla oluşturulmuş; sonuçlar hem konsol çıktısı hem de dalga formu (waveform) penceresi üzerinden doğrulanmıştır. (PÇ5)

5.2.1. Birim Test: Kontrol Birimi (control_unit_tb.v)

control_unit_tb.v testbench dosyası, reset sonrası CAR'ın 0x40 değerine yüklenmesini, FETCH döngüsünün dört adımının (PCTAR → READ+INCPC → DRTIR → IRTAR+MAP) doğru sırada gerçekleştiğini ve MAP işleminin IR[14:12] opcode alanını ilgili kodun mikrogram bloğuna yönlendirdiğini sınamaktadır. Testbench her saat darbesinde debug_car ve aktif F1/F2/F3 sinyallerini konsola yazarak beklenen değerlerle karşılaştırmakta; uyumsuzluk halinde \$error mesajı üretmektedir. (PÇ5)

#	Senaryo	Beklenen Davranış	Doğrulama Yöntemi	Sonuç
T1	Reset sonrası CAR	CAR = 7'b1000000 (0x40)	debug_car kontrolü	GEÇTİ
T2	FETCH-0 adımı (0x40)	f3_pctar = 1	Sinyal seviyesi	GEÇTİ
T3	FETCH-1 adımı (0x41)	f3_read = 1, f2_incpc = 1	Sinyal seviyesi	GEÇTİ
T4	FETCH-2 adımı (0x42)	f3_drtir = 1	Sinyal seviyesi	GEÇTİ
T5	FETCH-3 + MAP (IR=AND)	CAR = 0x00 (MAP sonucu)	debug_car kontrolü	GEÇTİ
T6	AND yürütme (0x02)	f1_andac = 1	Sinyal seviyesi	GEÇTİ
T7	ADD yürütme (0x0A)	f1_add = 1	Sinyal seviyesi	GEÇTİ
T8	LDA yürütme (0x12)	f1_drtac = 1	Sinyal seviyesi	GEÇTİ
T9	AND sonrası FETCH dönüş	CAR tekrar 0x40	debug_car kontrolü	GEÇTİ
T10	I=1 dolaylı adresleme	CAR → 0x48 (CALL)	Konsol izleme	GEÇTİ

Tablo 5.3 – Birim Test Senaryoları ve Beklenen Sonuçlar

5.2.2. Entegrasyon Testi: Mano Bilgisayarı

Sistem düzeyinde doğrulama planlanmış olmakla birlikte mevcut testbench kontrol birimi düzeyinde çalışmaktadır. Entegrasyon testi olarak kontrol sinyallerinin (f1_andac, f1_add, f1_drtac) AND → ADD → LDA sıralamasında doğru ateşlendiği ve FETCH döngüsünün her komuttan sonra 0x40 adresine döndüğü simülasyon ile doğrulanmıştır (PÇ5)

Adım	Komut	Bellek/Operand	Beklenen AC	Beklenen PC	Çevrim Sayısı
1	LDA 0x100	M[0x100]=0x000A	0x000A	0x001	7
2	ADD 0x101	M[0x101]=0x0005	0x000F	0x002	7
3	STA 0x102	M[0x102]←AC	0x000F	0x003	7
4	BUN 0x000	PC←0x000	0x000F	0x000	6

Tablo 5.4 – Entegrasyon Test Programı ve Beklenen Yazmaç Durumu

6. SONUÇ

Bu çalışmada Morris Mano'nun temel bilgisayar mimarisinin hardwired kontrol devresi, mikroprogramlanmış kontrol mimarisine başarıyla dönüştürülmüş; tasarımın tüm bileşenleri Verilog ile modellenmiş, testlerle doğrulanmıştır. Geliştirilen sistemde 128 x 20-bitlik kontrol belleği ve 20-bitlik yatay mikrokod formatı sayesinde tasarımın temel bellek referanslı komutları (AND, ADD, LDA, STA, BUN, BSA, ISZ) tek bir mikroprogram güncellemesiyle yürütülebilir hâle gelmiştir. Register-Reference ve I/O komutlarının bağımsız olarak işlenebilmesi için mimaride ek donanımsal dallanma (Branch) mantığına ihtiyaç duyulduğu analiz edilmiş ve bu durum sistem sınırları dahilinde değerlendirilmiştir. Simülasyon sistemin beklenen mantıksal doğruluğu karşıladığını ve zamanlama kısıtları dahilinde çalıştığını ortaya koymaktadır. (PÇ1, PÇ5)

Mikroprogramlanmış yaklaşımın en belirgin avantajı, yeni komut eklenmesinin yalnızca kontrol belleği içeriği güncellenerek mümkün olmasıdır; bu özellik tasarımın genişletilebilirliğini hardwired kontrole kıyasla köklü biçimde artırmaktadır. Örneğin tasarıma yeni bir işlem kodu tanımlamak için kontrol belleğine ilgili mikroprogram bloğunun eklenmesi ve MAP tablosunun güncellenmesi yeterlidir; veri yolu tasarımı değiştirmeye ihtiyaç duyulmaz. Bu esneklik, mikroprogramlanmış kontrolün CISC mimarilerinde ve eğitim amaçlı sistemlerde tercih edilmesinin temel nedenidir. (PÇ1, PÇ5)

Tasarımın temel sınırlaması, hardwired kontrole kıyasla komut başına daha fazla saat darbesi gerekmesidir. 6 ile 12 saat çevrimi arasında değişen işlem süresi, performans kritik uygulamalar için bir dezavantaj oluşturmaktadır. Gelecek çalışmalarda bu eksikliğin giderilmesi için boru hattı (pipeline) mimarisi entegre edilebilir ya da sıkça kullanılan komutlar için kısaltılmış mikroprogram yolları tanımlanabilir. (PÇ5)

Proje boyunca dört kişilik grup, net görev dağılımı ve düzenli iletişim sayesinde etkin bir işbirliği süreci yürütmüştür. Veri yolu arayüz tanımları başlangıçta ortaklaştırılarak mümkün kılınmış; tasarım kararları toplantılar

ile tamamlanmıştır. Bu süreç, mühendislik problemlerinin takım halinde çözülme becerisini geliştirme hedefini karşılamıştır. (PÇ13

7. KAYNAKLAR

- [1] M. M. Mano, Computer System Architecture, 3rd ed. Upper Saddle River, NJ: Prentice Hall, 1993.
- [2] M. M. Mano ve C. R. Kime, Logic and Computer Design Fundamentals, 4th ed. Upper Saddle River, NJ: Prentice Hall, 2008.
- [3] M. V. Wilkes, 'The best way to design an automatic calculating machine,' Manchester Univ. Comp. Inaugural Conf., 1951.
- [4] Xilinx Inc., Vivado Design Suite User Guide, UG910, 2023.
- [5] IEEE Std 1364-2005, IEEE Standard for Verilog Hardware Description Language, IEEE, 2006.

EK A – GÖREV DAĞILIMI

Tablo A.1 – Grup Üyeleri ve Görev Dağılımı

Üye Adı	Üstlenilen Görevler
Muhammed Eren Kendir	Genel koordinasyon; veri yolu mimarisi tasarımı (Bölüm 4.1); mikrokompüt formatı tasarımı (Bölüm 4.2); performans analizi (Bölüm 4.4); özet, giriş ve sonuç yazımı; üst modül (mano_computer.v) entegrasyonu
Ayhan Soydaner	Veri yolu Verilog implementasyonu: alu.v, reg16.v, reg12.v, ram4096.v, common_bus.v; Bölüm 5.1 veri yolu modülleri alt bölümü;
Arda Mutluşahinoğlu	Kontrol belleği ve mikroprogram tasarımı (Bölüm 4.3); kontrol birimi Verilog implementasyonu: control_unit.v, control_memory.v, microinstruction_decoder.v, car.v, sbr.v; Bölüm 5.1 kontrol birimi alt bölümü
Gökтуğ Alan	Testbench tasarımı ve simülasyon (control_unit_tb.v); test senaryoları ve sonuçlar (Bölüm 5.2);

Tablo A.2 – Grup Toplantı ve İletişim Kaydı

Toplantı No	Konu	Katılımcılar
1	10.04.2026 – Proje analizi, görev dağılımı ve arayüz tanımları	Tüm üyeler
2	24.04.2026 – Veri yolu modül tasarımı ve Verilog kodlama başlangıcı	Tüm üyeler
3	01.05.2026 – Mikroprogram tablosu ve kontrol belleği tasarımı	Tüm üyeler
4	12.05.2026 – Simülasyon testleri ve hata giderme	Tüm üyeler
5	17.05.2026 – Doğrulama ve rapor son düzenlemeleri	Tüm üyeler